

Incident Detection on Junctions Using Image Processing

Murat Tulgaç, Enes Yüncü, Mohamad-Alhaddad and Ceylan Yozgatligil

Abstract—In traffic management, it is a very important issue to shorten the response time by detecting the incidents (accident, vehicle breakdown, an object falling on the road, etc.) and informing the corresponding personnel. In this study, an anomaly detection framework for road junctions is proposed. The final judgment is based on the trajectories followed by the vehicles. Trajectory information is provided by vehicle detection and tracking algorithms on visual data streamed from a fisheye camera. Deep learning algorithms are used for vehicle detection, and Kalman Filter is used for tracking. To observe the trajectories more accurately, the detected vehicle coordinates are transferred to the bird's-eye view coordinates using the lens distortion model prediction algorithm. The system determines whether there is an abnormality in trajectories by comparing historical trajectory data and instantaneous incoming data. The proposed system has achieved %84.6 success in vehicle detection and %96.8 success in abnormality detection on synthetic data. The system also works with a %97.3 success rate in detecting abnormalities on real data.

Index Terms—Anomaly detection, Deep learning, Traffic management.

I. INTRODUCTION

THE demand for transportation rises year by year due to the increase in the population. As a result, traffic congestion occurs by exceeding the capacity of existing roads. Traffic congestions are examined in two groups as recurrent and non-recurrent ones. Recurrent congestions are based on a certain routine, and not affected by temporary events. Non-recurrent congestions are experienced due to weather or temporary events in traffic.

It is critical to inform the operators by identifying accidents, vehicle malfunctions, falling objects, or similar temporary events as soon as possible, especially at intersections. Fortunately, the study of smart monitoring systems has been a great interest in research and development. Remarkable progress has been achieved to deploy traffic monitoring systems that will detect abnormalities at intersections and notify who is in charge of taking action.

In this paper, we address the use of an omnidirectional (fisheye) camera for intersection monitoring. We provide a framework for incident detection which consists of three steps:

- 1) Detect vehicles in the camera Field of View (FoV)
- 2) Track the vehicles entering/exiting the FoV
- 3) Compare their followed paths trajectories with the normal path and detect any abnormality.

M.Tulgaç, E. Yüncü, and M.Alhaddad are with the ISSD, 06800 Ankara, Turkey (e-mail: murat.tulgac@issd.com.tr; enes.yuncu@issd.com.tr; muhammed@issd.com.tr).

C.Yozgatligil is with the Department of Statistics at METU, 06800 Ankara, Turkey (e-mail: ceylan@metu.edu.tr).

In order to detect an anomaly, normal behavior of the specific route must be characterized [1]. Afterward, by monitoring the system behavior continuously, the anomaly can be detected by defining time intervals that do not comply with normal behavior. In the proposed method, the difference between the normal behavior and the behavior of the entering vehicles is evaluated on the trajectories followed by the vehicles. For trajectory extraction, we utilized deep learning techniques to develop our vehicle detector, then we mapped the scene to the bird's-eye view. Finally, we tracked the vehicles using Munkres and Kalman filter methods.

In this study, we propose a framework that detects trajectories on the images taken from a fisheye camera installed at the intersection and detects abnormalities by comparing these trajectories with the past trajectories. This framework acts as a subsystem for Intelligent Traffic System (ITS). The flow chart of the proposed method is shown below in Figure 1.

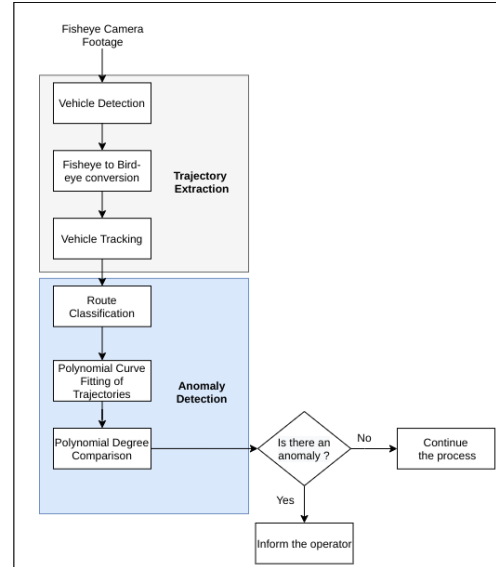


Fig. 1: Flow chart of the proposed system.

The rest of the study is evaluated under four headings: Section-2 addresses the literature advancement in incident detection at intersections. Section-3 illustrates the synthetic and real data used in the study. In the Section-4, the proposed method is explained and sub-steps related to the method are presented. In Section-5, the results of the study on synthetic and real data, conclusions, and summary of the study are presented.

II. RELATED STUDIES

The omnidirectional cameras are known for their wider Field of View (FoV) than conventional cameras. Thus, using them in monitoring tasks is beneficial in terms of setup and maintenance costs. The studies addressing omnidirectional cameras have reported several challenges faced in capturing a large image and analyze the highly distorted objects at variable scales in real-time.

In their work, Lee et al. [2] have created a vehicle trip table estimation which tracks the origin-destination points of vehicles passing through the intersection. Their tracker relies on particle filters and motion dynamics and has suffered from appearance changes and different weather conditions which makes it limited in robustness. Wang et al. [3] have addressed the issues faced by relying on a fisheye camera for vehicle counting at an intersection, due to severe distortion and perspective effects. They came up with a method based on motion similarity and probabilistic weights to transfer the motion knowledge between long and short-term trajectories between points. Other attempts to surmount the distortion was to track motion based on background subtraction models [4, 5, 6].

Other studies have addressed the same problem with standard cameras (monocular vision), and remarkable outputs were reported so far in the literature. We found out that the researchers usually attempt to do background subtraction or feature extraction and tracking variants at the first step for detection purposes, then tracking the vehicle path with other perimeters [7, 8, 9, 10, 11].

To track the behavior, some studies attempt to rely on the vehicle size ratio in correlation with its direction, velocity, and acceleration obtained from the detection step [12] to understand the onsite situation, while others rely on kernel-based filters to determine whether the distribution of the detected pixels represents an accident or not [13].

The main difference between existing attempts and our approach is the lack of incident detection exclusiveness. The current studies focus on the situation where two vehicles may collide, while our approach can detect abnormality based on the passing vehicles' trajectories characteristics. Another aspect is that we embed deep learning trends for ITS applications where addressed research has relied heavily on background subtraction which suffers in terms of robustness, especially for omnidirectional vision.

III. DATA

Two different data are used in the development and testing phases of the algorithm. These are simulation and real data of vehicles passing the intersection. An algorithm has been developed in which accurate results can be obtained for anomaly detection in both real and simulated data.

A. Simulation Data

Simulation data is obtained from the simulation program called VisSim [14]. This data is used in the creation of deviation detection algorithms in trajectory. ComAPred to the fisheye dataset, simulation data has a lower noise rate

(Figures 9-10). This caused the trajectories of the vehicles in the simulation data to be much smoother than the actual data. Because of that, new tests were carried out on the fisheye dataset after implementing and testing the algorithm on simulation data to develop a more robust process.

Simulation data is created by simulating the 1-hour flow of a junction of a city in Turkey that is used while the fisheye camera dataset is created. Seven different scenarios (incident at different specific routes, no incidents) are created to test how the system will respond to different situations. In addition to these different scenarios, to characterize the normal behavior, the simulation scenario with no incident was run 30 times. Hence, 37 hours of simulation data are used. In this data, as with tracking algorithms, vehicle-based coordinate data are obtained, and consistency with real data is achieved. The simulation produces several vehicles $V_1, V_2 \dots V_n$ passing through the junction with different followed paths. Each followed path is sampled over several subsequent locations $L_{x,y}$ from the junction entrance until exit. (x, y) are the Cartesian coordinates of the vehicle's V location L .

After the vehicle-based coordinate data is normalized on all data, the trajectories of the vehicles are sorted by time, and the differences from previous trajectories are observed. Related images for the intersection used in the study and simulation data are shown in Figure 2.

B. Fisheye Dataset

We had the access to the installed fisheye surveillance cameras mounted at 8 meters height from the ground on junctions from a city in Turkey. The coverage of the camera reaches a clear human observable view of around 75 meters from the junction entrance point. Each road can accommodate 3 lanes of vehicles that vary in size (car, truck, bus). The obtained dataset [15] is acquired during daytime and sampled at 10fps frames, resulting in 20,000 frames, a sample of the mentioned data is in Figure 3.

IV. PROPOSED METHOD

A. Vehicle Detection

Vehicle detection is a variant of the classic object detection problem in computer vision research. Several approaches have addressed detecting vehicles with different study scopes such as autonomous vehicles [16, 17, 18, 19, 20, 21, 22, 23, 24, 25] and smart cities models [26, 27, 28, 29, 30, 31, 32, 33, 34]. In our proposed framework, the first step is to detect vehicles at the junction, the capturing is via a static camera with a fisheye lens.

The fisheye lens has introduced a distortion problem in the vehicle shapes which would fail several techniques used in the research, where standard cameras were used. Some studies tried to overcome the distortion issue with different techniques. In one approach [35] they have trained a network over the mapping between a fisheye captured lines image and straight lines rectified image to overcome that issue, then let the network auto-rectify any new fisheye incoming image. Affine transformations have also proved to be a solution to overcome the lens effect [36, 37]. However, it is considered

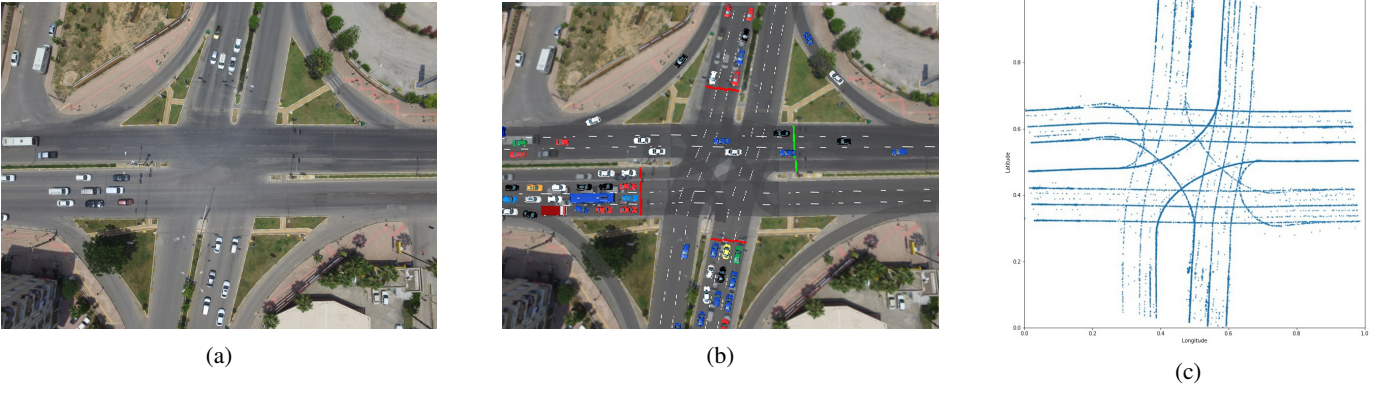


Fig. 2: Intersection footage. (a) Drone recorded image of the intersection used in the study. (b) Simulated image of the intersection. (c) Routes in the simulation data.

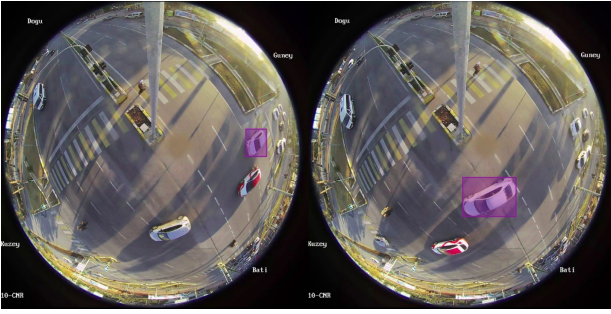


Fig. 3: Example of the fisheye data.

to be a computationally expensive operation as a pre-process [38]. Another study [39] suggests working on the camera calibration itself. Although the found literature is massively informative, the nature of the open environment has limited us from having costly detection pre-processing computation or hardware calibration.

Here we follow the “use it as it exists” approach to come up with a detection algorithm. We target the fisheye camera as the feed source with no preprocessing taking place, thus eliminating any deployment ambiguities of configuring the algorithm at the post-training stage.

The above-mentioned fisheye data is splitted to train-test subsets 80% by 20% respectively. We trained several models on the raw images and tabulated the results in Table-1. The model configuration is cloned from [40]. It is found that model YoloV3 [41] overperforms YoloV3-tiny and the Faster-RCNN in the defined scores. Thus, it is selected for the vehicle detection stage. It is well known that relying on deep learning solutions for detection would leverage robustness, but penalize computation time and resources allocation [42].

To evaluate the model, we defined several parameters. For measuring how much punctual the model is we used PASCAL [43] average precision formula at 3 different Jaccard index (a.k.a IoU) thresholds of interest 50, 75 and 95 %. The thresholds are picked to get a better understanding of model performance since any vehicle’s size in fisheye camera changes massively. For the inference time measurement, the runtime CPU is INTEL core i7-8750H. Due to the existence of several

parameters, we defined a formula (Equation-1) to pick up the best model, as illustrated in Table-1.

$$\begin{aligned} score = & 0.4 * mAP_{IoU50\%} \\ & + 0.3 * mAP_{IoU75\%} \\ & + 0.1 * mAP_{IoU95\%} \\ & + 0.2 * (1 - inference) \end{aligned}$$

TABLE I: Vehicle detection models

Model	$mAP_{IoU(50\%)}$	$mAP_{IoU(75\%)}$	$mAP_{IoU(95\%)}$	$Inference(msec)$	$score(\%)$
YoloV3	95.6	89.9	80.7	250	88.28
YoloV3-tiny	89.5	80.9	55	80	83.97
Faster-RCNN	85	75	70	420	75.1

B. Fisheye to Bird’s-eye View Conversion

The wide field of view (FoV) of a fisheye camera benefits in better image coverage compared to a standard surveillance camera [36, 44]. After the vehicles are detected in the fisheye view, it is required to obtain vehicles’ trajectories. To limit the computation resources required for this step, it is favored to do a simple pixel transformation mapping on the initial frame and use the obtained transformation parameters to project the trajectories into the bird’s-eye view. To do so, as a post-processing step, a geometrical rectification procedure takes place as follows:

- 1) Define $L_f(x_c, y_c)$ as the center point (in pixels) of the vehicle’s V bounding box resulted from the detector at fisheye view
- 2) Manually dictate the initial frame key points for rectification to bird eye view
- 3) Obtain the transformation map matrix
- 4) Apply the transformation map matrix against vehicle location L_f to get L_b as the vehicle V location at the bird’s-eye view; we denote L_b as our L for the sake of simplicity.

The algorithm of distortion model estimation is described in Equation-2:

$$D(P_f, P_c) = K(z) * D(P_b, P_c). \quad (1)$$

where $D(a,b)$ is defined as $\begin{pmatrix} a_x - b_x \\ a_y - b_y \end{pmatrix}$. P_c is defined to be the center point of distortion which is the center of the image frame, P_f is the point P in the fisheye view while P_b is the point P in the bird's-eye view. In order to obtain the trajectory point P_b of P_f it is required to solve the Equation-3 for $K(z)$ where $K(z)$ is the function approximating the distortion as a Taylor series defined as:

$$K(z) = q_0 + q_1 z + q_2 z^2 + q_3 z^3 + \dots \quad (2)$$

The z is the Euclidean distance between P_c and P_f :

$$z = \sqrt{(P_{cx} - P_{fx})^2 + (P_{cy} - P_{fy})^2}. \quad (3)$$

Solving for K requires obtaining the distortion parameters set $(q_0, q_1, q_2, \dots)$. This is done at the initially obtained frame, where a line of points P_f is drawn on the fisheye image following the path of the street, as in Figure 4. Lastly using the obtained distortion parameters we are able to project each vehicle location L_f to L_b .

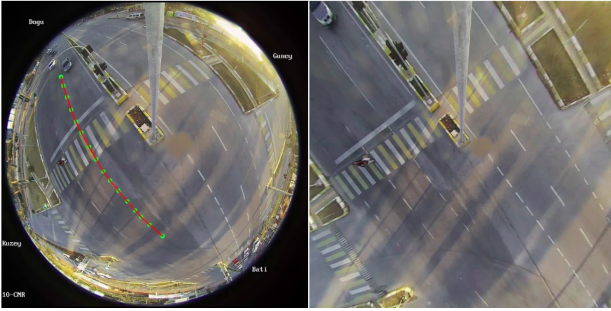


Fig. 4: On the left is a fisheye frame having a drawn line of points P_f is defined to follow the street path, on the right is the rectified bird's-eye version of the frame

C. Tracking the vehicle trajectory

To define the vehicle trajectory it is required to track each passing vehicle throughout the junction. Research community has addressed object tracking problem with geometrical algebra [45, 46, 47, 48, 49] as well as recent deep learning-based solution [50, 51, 52]. The requirement to have a lightweight solution for trajectory extraction emphasizes going with the Munkres [53] Kalman [54] combination, thus achieving high accuracy and computationally efficient solution.

Several studies have addressed object association using the Munkres algorithm. In the study of body parts association of COCO 2016 challenge Cao et al. have suggested the Munkres algorithm for optimal results [46]. Others like Mozaffari et al. have used it in Unmanned Autonomous Vehicle (UAV) deployment and association problems in order to keep track of the devices [47].

The Munkres is to link identified objects at frame F_{t-1} and frame F_t . To utilize it in our vehicle association and have a track of the vehicles' Spatio-temporal status, we have favored the conditional context to be the Jaccard Index score. Although

some addressed studies have preferred to use Euclidean distance [48] the fact of vehicles can get too close may trick the algorithm at that point.

$$J_{t-1,t} = \begin{pmatrix} j_{1t-1,1t} & j_{1t-1,2t} & j_{1t-1,3t} \\ j_{2t-1,1t} & j_{2t-1,2t} & j_{2t-1,3t} \\ j_{3t-1,1t} & j_{3t-1,2t} & j_{3t-1,3t} \end{pmatrix} \quad (4)$$

where $j_{t-1,t} = \text{Jaccard}(V_{t-1}, V_t)$ and V is the vehicle detected at location L in the previous step. The method will associate the vehicles by obtaining the maximum IoU $\text{Max}(J_{t-1,t})$, as illustrated in Figure 5.

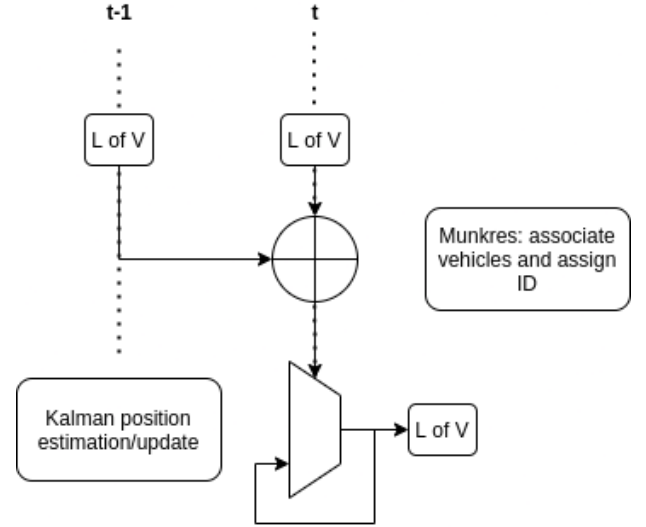


Fig. 5: Vehicle V trajectory tracking procedure

After assigning the vehicles, it is required to track their path throughout the junction. The more the path tracking mechanism is consistent, the better the proposed method performance. Kalman filter has proved to be a state of the art tracking method, deployed in several studies [48, 55, 56, 57]. We use Kalman filter to estimate vehicle V next location L_t based on L_{t-1} and smooth the trajectory produced in case the detector failed to detect V at a frame F . The vehicle location L is defined for the tracker when the V enters the camera FoV and is dismissed when it leaves the FoV. The output of this stage can be formulated as below:

$$\hat{V} = \text{Kalman}(\text{Munkres}(V_{@L})) \quad (5)$$

where $V \in \{V_{0@L0}, V_{1@L1}, \dots\}$ and $\hat{V}$ denotes the set of tracked vehicles in the scene at time t .

D. Route Classification

At the initial stage, each route in the junction is drawn manually over the image frame. The route line consists of 8 to 10 connected points are fitted to 2 to 7-degree polynomials as shown in Figure 6-a and 6-b.

After that, the mean absolute errors are calculated to correlate the vehicle V 's trajectory with the defined routes, with the formula in Equation-7. Finally, the vehicle V belongs to the route with the lowest error.